BTS SERVICES INFORMATIQUES AUX ORGANISATIONS **SESSION 2026**

Conception et développement d'applications (option SLAM)

DESCRIPTION D'UNE APPLICATION PROFESSIONNELLE

N° réalisation : 1

Nom, prénom : Juhasz Klaudia | **N° candidat :**

Épreuve ponctuelle ☒ **Contrôle en cours de formation** ☐ | **Date :** 20 / 06 /2025

Organisation support de la réalisation professionnelle
IRIS MediaSchool Nice

Intitulé de la réalisation professionnelle
ClassCord - Client de messagerie interopérable

Période de réalisation : 16/06/2025 – 20/06/2025 **Lieu :** Nice
Modalité : ☒ **Seul(e)** ☐ **En équipe**

Compétences travaillées
☒ Concevoir et développer une solution applicative
☒ Assurer la maintenance corrective ou évolutive d'une solution applicative
☒ Gérer les données

Conditions de réalisation[1] (ressources fournies, résultats attendus)

Ressources fournies
- Contexte et cahier des charges fonctionnel et technique détaillé.
- Serveur de test opérationnel mis à disposition par le formateur.
- Dépôt GitHub modèle à forker pour démarrer le projet.
- Accompagnement pédagogique tout au long de la semaine.

Résultats attendus :
Une application cliente fonctionnelle permettant :
- Connexion à un serveur de chat.
- Envoi/réception de messages globaux et privés.
- Gestion des statuts utilisateurs.
- Interface Swing fluide et conviviale.
- Communication en JSON avec le serveur.

Description des ressources documentaires, matérielles et logicielles utilisées[2]

Ressources documentaires
- Cahier des charges fourni par le formateur et documentation Maven & org.json.
- Cours Java (Sockets, Swing, Threads).
- Aide-mémoire Markdown pour le README et tutoriels en ligne (Java Swing, communication réseau).

Ressources matérielles
- Poste informatique personnel ou mis à disposition en salle BTS SIO.
- Connexion au réseau local pour tests avec serveur et clients.

Environnement logiciel
- Système d'exploitation : Windows 11
- IDE principal : Visual Studio Code (avec extensions Java) et IntelliJ IDEA
- Gestionnaire des dépendences : Maven
- Dépendance externe : org.json:json:20231013
- Contrôle de version : Git et plateforme GitHub
- Protocole de communication : Sockets TCP avec messages JSON (terminés par \n)
- Langage de programmation : Java 11 ou supérieur

Modalités d'accès aux productions[3] et à leur documentation[4]
Le projet est disponible sur mon dépôt GitHub à l'adresse suivante : https://github.com/juklau/Classcord-client.git .
La documentation du projet est fournie à travers les fichiers `README.md` et `README_V1.md`, qui décrivent la conception et les fonctionnalités de l'application: https://github.com/juklau/Classcord-client/tree/main/classcord-client/docs
Les instructions de lancement sont détaillées dans la section **« Installation et lancement »** du fichier `README.md`.

---

[1] En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

[2] Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

[3] Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

[4] Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

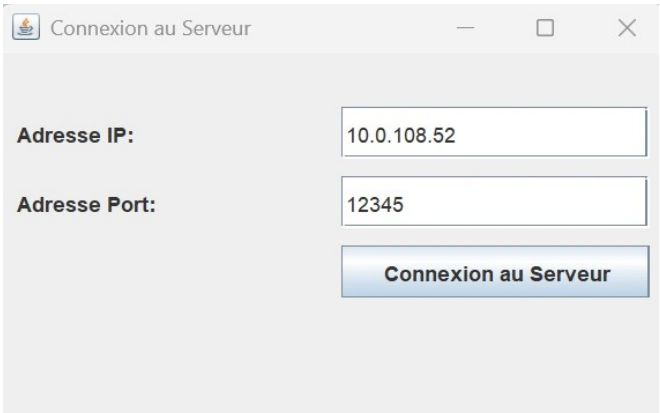
Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Contexte et objectifs

Dans le cadre de la semaine intensive SLAM, j'ai participé au développement de **ClassCord-client**, une application Java Swing de messagerie instantanée en réseau local. Cette application cliente interagit avec un serveur TCP (fourni ou géré par les étudiants SISR) via des échanges JSON pour offrir une interface fluide, sécurisée et réactive aux utilisateurs (étudiants et enseignants).

Architecture mise en œuvre

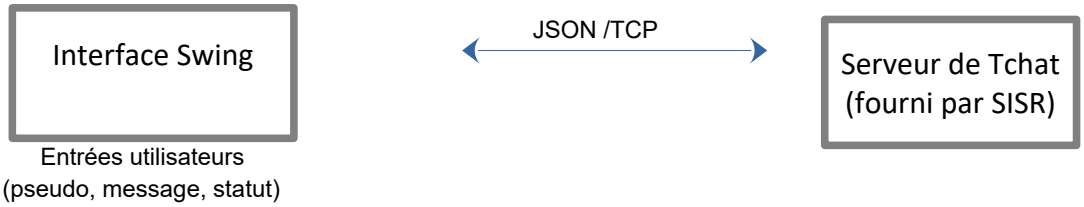
```
├── src/ main/ java/ fr/ classcord/
│   →   →   →   ├── controller/ AuthController
│   →   →   →   │   →   ├── ChatController
│   →   →   →   │   →   ├── LoginController
│   →   →   →   │   →   ├── SessionController
│   →   →   →   ├── model/ ClientInvite
│   →   →   ...  │   ├── CurrentUser (inactif)
│   →   →   →   │   ├── User.java (inactif)
│   →   →   →   │   ├── Message.java (inactif)
│   →   →   →   │   ├── UserColorManager
│   →   →   →   ├── ui/  ChatPersoUI
│   →   →   →   │   ├── ChatUI
│   →   →   →   │   ├── ChoixModeUI
│   →   →   →   │   ├── ConnectToServeurUI
│   →   →   →   │   ├── GuestUI
│   →   →   →   │   ├── LoginUI
│   →   →   →   │   ├── UserStatusRenderer
│   →   →   →   ├── app/ App (inactif)
├── pom.xml
```



Fonctionnalités développées

- Connexion à un serveur distant (saisie IP/port)
- Connexion en tant qu'invité ou avec authentification (login/mot de passe)
- Affichage des messages globaux en temps réel
- Envoi de messages privés avec distinction visuelle
- Affichage dynamique de la liste des utilisateurs connectés avec leur statut
- Modification de son propre statut (Disponible / Absent / Invisible / Indisponible)
- Interface graphique fluide avec gestion des threads réseau
- Gestion des erreurs (déconnexion propre, identifiants invalides, etc.)

Schéma fonctionnel simplifié



Productions réalisées

- Projet Maven complet avec structure professionnelle
- Classes Java métier (User, Message) + logique réseau encapsulée
- Interface Swing moderne, intuitive et modulaire
- Gestion multithreadée pour réception asynchrone
- README.md avec instructions de lancement et captures
- Documentation intégrée (commentaires, structure MVC)

Tests et validation

- Tests multi-clients avec interaction serveur réel
- Vérification des statuts, messages privés (MP), messages globaux
- Démonstration complète : connexion, tchat, changement de statut, déconnexion propre

Projet mené de bout en bout dans un dépôt GitHub personnel avec commits réguliers et ce projet m'a permis de mobiliser concrètement mes compétences en Java, Swing, réseau, MVC et gestion de projet.